Bryan - Project Portfolio Page

Overview

Business Management System (BMS) is a Java command-line desktop application designed for small, growing retail businesses. It consolidates product catalogue management, physical inventory tracking, customer orders, sales revenues, and operating expenses into a single offline-capable terminal interface.

My primary contribution was designing and developing the **Stock & Inventory Management Subsystem**, including stock adjustments, low stock alerts, cross-subsystem order synchronisation, core Gradle build infrastructure, and architectural documentation.

Summary of Contributions

- **Code contributed**: tP Code Dashboard

- **Feature: Physical stock inventory management** (`stock add`, `stock remove`, `stock list`)

  - What it does: tracks available physical inventory for catalogued products, handles restocking (`stock add`), inventory deductions (`stock remove`), and tabular overview displays (`stock list`).

  - Justification: separates product catalogue definitions (name, master price) from operational stock counts, allowing stock to be adjusted for deliveries, damage, or audits without altering catalogue data.

  - Highlights: refactored procedural CLI handling into an extensible OOP Command pattern (`AddStockCommand`, `RemoveStockCommand`, `ListStockCommand`). Engineered `stock remove` with safe boundary clamping (`Math.max(0, currentStock - quantityToRemove)`) and a default of 1 unit when `q/` is omitted, guaranteeing inventory never drops below zero. Routed parsing through `Parser` with case-insensitive matching and whitespace trimming, while delegating dynamic table formatting to `Ui`.

- **Feature: Low stock monitoring** (`stock low [q/THRESHOLD]`)

  - What it does: alerts users to products nearing depletion by filtering inventory items whose available quantity is at or below a specified threshold (defaulting to 5 units).

  - Highlights: supports an optional threshold specified either with a `q/` prefix (e.g. `stock low q/10`) or as a plain number (e.g. `stock low 10`), defaulting to 5 if omitted. Implemented input validation rejecting negative or non-integer thresholds, defensive quantity reconciliation with `Product.getStock()`, and dynamic tabular formatting with informative status messages when all stock is sufficient or empty. Registered under `help stock` (#70).

- **Testing and diagnostics**:

  - Authored comprehensive JUnit 5 test suites in `StockCommandTest`, `UiTest`, and `ParserTest`, testing restock additions, depletion clamping, uncatalogued product rejections, and UI tabular alignments.

  - Implemented regression tests for Issue (#68), verifying stock deductions during `order add`, restorations during `order cancel`, adjustments during `order edit`, and assertion validations for null/negative domain invariants.

  - Added unit tests for default and custom thresholds, empty lists, and sufficient stock messages for `LowStockCommand`.

- **Documentation**:

  - User Guide: authored Section 2 (Stock inventory: `stock add`, `stock remove`, `stock list`, `stock low`), documenting syntax, parameter constraints, validation rules, and example outputs. Updated the Command Summary table to include `stock low`. Refined Introduction, Quick Start, and FAQ sections.

  - Developer Guide: authored Section 2.4 (Stock and inventory management), documenting class structure, domain models, cross-subsystem order synchronisation, and design considerations/alternatives. Created and maintained Mermaid UML diagrams: class diagram (`stock-class.mmd`) and sequence diagram (`stock-low-sequence.mmd`). Authored Section 4 (User Stories using SE-Education star priorities), Section 5 (Non-Functional Requirements), Section 6 (Glossary), and Section 7 (Manual testing guide).

- **Contributions to team-based tasks**:

  - Updated team profile in `docs/AboutUs.md`.

  - Resolved merge conflicts and cross-branch build failures involving stock commands, help messages, and test regressions.

  - Formatted and structured Developer Guide sections to adhere to course standards.

- **Review/mentoring contributions**:

  - Reviewed and merged customer & sales feature PRs: customer payment tracking (#64), customer order details (#65), help command system (#66), consecutive sales numbering fix (#67), receipt generation (#26), and sales listing (#27).

  - Reviewed and merged product catalogue PRs: product domain assertions (#47) and product catalogue User Guide documentation (#48).

  - Assisted team members with Gradle wrapper execution errors on Windows paths (`gradlew.bat`), build initialisation, and cross-IDE Java 25 toolchain configuration.

- **Contributions beyond the project team**:
  - *(To be updated: bugs reported during Practical Exam Dry Run / forum contributions)*.